Cairo University
Faculty of Engineering
Department of Computer Engineering

Accelera

A Graduation Project Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the Requirements for the Degree
of
Bachelor of Science in Computer Engineering.

Presented by

Mohamed Ashraf
Nesma Osama
Mazen Adel
Mohamed Khater

Supervised by

Dr. Salma Abdel Monem

April 17, 2026

Abstract

Accelera is a hybrid Python/C++ machine learning framework that combines a high-level Python interface with native execution components. The system is organized around five modules: Accelera Pipe, the Code Parallelizer, AutoPreprocessing, AutoML, Deployment, and the Benchmark Platform. Accelera Pipe represents machine-learning workflows as directed graph nodes for preprocessing, modelling, prediction, metric evaluation, branching, and merging. The Code Parallelizer analyzes C/C++ and supported Python code, extracts loop features, predicts parallelization opportunities, and emits OpenMP-annotated C++ code. This report documents the design, methodology, and experimental results available for the implemented modules, while retaining the required report structure for the remaining modules.

Arabic Abstract

Acknowledgment

Table of Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
AST	Abstract Syntax Tree
CPU	Central Processing Unit
ML	Machine Learning
OpenMP	Open Multi-Processing
RSS	Resident Set Size
SVC	Support Vector Classifier
VMS	Virtual Memory Size

List of Symbols

X	Input feature matrix
y	Target vector
P	Source program
L	Extracted loop
$F(L)$	Feature representation of loop L

Contacts

Team Members

Name	**Email**	**Phone Number**
Name of Student 1	abc1@email.com	+2 01xxxxxxxx
Nesma Osama	nesma.ahmed03@eng-st.cu.edu.eg	+2 01067193062
Name of Student 3	abc1@email.com	+2 01xxxxxxxx
Name of Student 4	abc1@email.com	+2 01xxxxxxxx

Supervisor

Name	**Email**	**Number**
Salma Abdelmoniem	abc1@email.com	+2 01114302362

Chapter 1: Introduction

1.1 Motivation and Justification

Accelera addresses the practical gap between the productivity of Python and the performance requirements of machine-learning workflows and loop-heavy data processing. A sequential script evaluates alternatives one after another and leaves memory lifetime and parallel execution decisions to the user. The project provides a graph-based pipeline backend and a code parallelization path to make these execution decisions explicit and reusable.

1.2 Project Objectives and Problem Definition

The project objective is to provide an integrated framework for constructing, executing, evaluating, and accelerating data-science workflows. The problem is to preserve a convenient Python-level programming model while enabling native graph scheduling, controlled intermediate-data lifetime, and OpenMP-backed execution for supported custom code.

1.3 Project Outcomes

The project is organized into the following modules:

- Accelera Pipe for graph-based machine-learning workflows.
- Code Parallelizer for source analysis and OpenMP generation.
- AutoPreprocessing for tabular, text, and image data preparation.
- AutoML for model selection and optimization.
- Deployment and Benchmark Platform components for serving and evaluation.

1.4 Document Organization

Chapter 2 reserves the required market visibility study. Chapter 3 contains the literature-survey structure. Chapter 4 documents the system design and implemented module methodology. Chapter 5 presents testing and verification results. Chapter 6 concludes the report and records future work. The appendices retain the required development, user-guide, and feasibility-study sections.

Chapter 2: Market Visibility Study

In this chapter, we will identify Accelera's target customers, talk about similar tools and platforms in the market, discuss the weaknesses and gaps in current solutions, and present the business case and financial analysis of the system.

2.1 Targeted Customers

Our targeted customers are data scientists and machine learning engineers, both beginners and experts. Accelera aims to support these users by providing a tool that automates and accelerates several steps in the data science life cycle. This helps users reduce repetitive manual work, and save time.

2.2 Market Survey

Many existing platforms provide useful features for data science and machine learning development, but these features are often available in separate tools or are limited to specific parts of the workflow. However, Accelera aims to provide these features within one toolkit.

2.2.1 AutoCleanML

AutoCleanML is a Python tool that focuses mainly on automating data cleaning and preprocessing tasks for machine learning datasets.

Pros:

- Makes ML data preprocessing automatic and intelligent
- Provides a report that explains the preprocessing decisions made by the tool.

Cons:

- Its scope is mostly limited to data cleaning and preprocessing.
- It does not support preprocessing for image and text datasets.

2.2.2 Kaggle

Kaggle is an online platform where the data science and machine learning community comes together. It is best known for its competition platform.

Pros:

- It introduces competitions.

- It ranks users in competitions.
- It has many public datasets.

Cons:

- It depends mainly on notebook-style work, so projects can become difficult to organize.
- Kaggle mainly focuses on competitions, and datasets, rather than providing a complete machine learning workflow in one toolkit.

2.3 Business Case and Financial Analysis

Chapter 3: Literature Survey

In this chapter, we provide an overview of the topics, algorithms and techniques needed to understand the project as a whole. First, we discuss the technical topics and background knowledge.Then we summarize related papers to our project.

3.1 Background and Previous Work

This section explains the main concepts and techniques related to the Accelera modules.

3.1.1 Data Science

Data science is a field that extracts knowledge and insights from data using scientific methods, processes, and algorithms.

3.1.2 Machine Learning

Machine learning is a field of artificial intelligence that allows computers to learn patterns from data without being explicitly programmed for every rule. A machine learning model is trained using examples, and then it uses what it learned to make predictions on new data.

3.1.3 Supervised Machine Learning

In supervised learning, the dataset contains input features and a known target output.

3.1.4 Classification

Classification is a supervised machine learning task where the output is a class or category. For example, a model can classify a medical case as positive or negative, or an image as one of several object classes.

3.1.5 Regression

Regression is a supervised machine learning task where the output is a continuous numerical value. For example, a model can predict house prices.

3.1.6 Natural Language Processing NLP

Natural Language Processing is a field that focuses on enabling computers to understand and process human language. Text data cannot be used directly by most machine learning models, so it must first be cleaned and converted into numerical

features.

3.1.7 Exploratory Data Analysis

Exploratory Data Analysis is the process of analyzing and visualizing the dataset to understand its main properties. It can include summary statistics, missing value counts, class distribution, feature distributions, and correlation between numerical features.

3.1.8 Data Preprocessing

Data preprocessing is the stage of transforming raw data into a clean and usable format for machine learning models. Raw datasets often contain missing values, duplicated rows, categorical text values, different numerical ranges, noisy text, or images with different sizes. These problems must be handled before training a model.

3.1.9 Missing Value Imputation

Missing value imputation is a preprocessing technique used when some values in a dataset are empty or unknown. Instead of removing all rows that contain missing values, the missing values can be replaced using a suitable value.

3.1.10 Robust Scaling

Robust scaling is a scaling technique that uses the median and the interquartile range instead of the mean and standard deviation. This makes it useful when the data contains outliers, because extreme values affect the median less than they affect the mean.

$$x' = \frac{x - \mathrm{median}(X)}{\mathrm{IQR}(X)}$$

where x' is the scaled value, x is the original value, $\mathrm{median}(X)$ is the median of the feature values, and $\mathrm{IQR}(X)$ is the interquartile range.

3.1.11 Categorical Encoding

Categorical encoding is the process of converting text categories into numeric values. Many traditional machine learning models and preprocessing pipelines expect numerical input.

3.1.12 Label Encoding

Label encoding is a simple encoding technique that converts each category into a number.

3.1.13 Binary Encoding

Binary encoding is a categorical encoding technique that first gives each category a number, then converts this number into binary form. The binary digits are then stored as separate columns.

3.1.14 Frequency Encoding

Frequency encoding replaces each category with how often it appears in the dataset. If a category appears many times, it receives a higher frequency value.

$$f(c) = \frac{\text{count}(c)}{N}$$

where $f(c)$ is the frequency of category c, $\text{count}(c)$ is the number of rows containing this category, and N is the total number of rows.

3.1.15 Term Frequency-Inverse Document Frequency TF-IDF

It is a technique used to convert text into numerical features by giving each word a weight based on how important it is in a document and across the whole dataset. Term Frequency measures how often a word appears in a document. Inverse Document Frequency gives lower weight to words that appear in many documents, because these words are usually less useful for classification.

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

where $TFIDF(t, d)$ is the weight of term t in document d, $TF(t, d)$ is the frequency of the term in the document, and $IDF(t)$ measures how rare the term is across all documents.

3.1.16 Image Preprocessing

Image preprocessing prepares image data before it is used by a deep learning model. Images may have different sizes, formats, or pixel ranges, so they need to be converted into a consistent form. Common image preprocessing steps include loading images, validating that they are readable, resizing them, and normalizing pixel values.

Pixel normalization usually converts image values from the range $[0, 255]$ to a smaller range such as $[0, 1]$ using:

$$x' = \frac{x}{255}$$

where x is the original pixel value and x' is the normalized value.

3.1.17 Classification Evaluation Metrics

Classification metrics are used to evaluate the performance of a classification model. Accuracy measures how many predictions are correct out of all predictions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP is true positives, TN is true negatives, FP is false positives, and FN is false negatives.

Precision measures how many predicted positive samples are actually positive:

$$Precision = \frac{TP}{TP + FP}$$

Recall measures how many actual positive samples were correctly detected:

$$Recall = \frac{TP}{TP + FN}$$

The F1-score combines precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3.1.18 Regression Evaluation Metrics

Regression metrics are used when the model predicts continuous numerical values. Mean Absolute Error measures the average absolute difference between the real values and predicted values:

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

where y_i is the actual value and $\hat{y}_i$ is the predicted value.

Mean Squared Error gives more penalty to large errors:

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

3.2 Comparative Study of Previous Work

3.2.1 Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices [1]

This paper discusses the role of data preprocessing and feature engineering in machine learning and data mining projects. It explains that preprocessing is not only a preparation step before model training, but also an important part of the complete

machine learning pipeline. The paper presents a general preprocessing framework that includes data cleaning, data transformation, feature construction, feature selection, and dimensionality reduction. For each stage, it explains different techniques and discusses the advantages and disadvantages of each one.

3.3 Implemented Approach

3.3.1 Auto Preprocessing

Based on the techniques reviewed in the previous section, this project implements an automated preprocessing approach for tabular datasets. The approach was designed to prepare data for machine learning models by applying suitable preprocessing steps automatically according to the type and characteristics of each column.

The general framework of the approach is as follows:

1. Load the dataset and define the target column.
2. Remove duplicate records.
3. Split the data into training and validation sets.
4. Analyze each column using data type, missing percentage, unique values, and basic statistics.
5. Drop unsuitable columns, such as constant columns or columns with missing values above the selected threshold.
6. Detect column types automatically.
7. Apply missing-value imputation.
8. Apply categorical encoding.
9. Apply robust scaling.
10. Apply feature selection using mutual information.
11. Save the preprocessing pipeline and generate a report.

The chosen approach follows a classical automated preprocessing pipeline. The main preprocessing decisions are summarized as follows:

- **Missing values:** Median imputation is used for numerical features because it is simple and more robust to outliers than mean imputation. For categorical features, most-frequent imputation is used because it replaces missing values with valid existing categories.

- **Numerical scaling:** `RobustScaler` is used for numerical features because tabular datasets may contain outliers or skewed values. It depends on the median and interquartile range, which makes it more stable than standard scaling when extreme values exist.
- **Categorical encoding:** The encoding method is selected according to the feature type and cardinality. Binary columns are encoded using ordinal encoding, low-cardinality categorical columns are encoded using binary encoding, and high-cardinality categorical columns are encoded using frequency encoding. This helps reduce the number of generated features and avoids high dimensionality.
- **Feature selection:** Mutual information is used because it is a filter-based method that can work with both classification and regression tasks. It helps keep the most informative features and remove weak features before model training.

This approach was chosen because it gives a good balance between automation, simplicity, speed, and interpretability. More advanced techniques, such as K-nearest neighbour imputation, autoencoders, and wrapper feature selection, can be useful in specific cases, but they need more computation and tuning. For this reason, classical preprocessing techniques were selected because they are more suitable for a general automated tabular preprocessing framework.
Another important addition in this project is that the module is not only used for data preparation. It also supports exploratory data analysis , where the generated report helps the user explore and understand The report includes useful information such as dataset size, column types, missing values, duplicates, visualizations, and the preprocessing steps applied.
In addition, the module was extended to support text and image data preparation, not only tabular data. For text data, the system cleans the text and converts it into numerical features for classification. For image data, the system loads, validates, resizes, and normalizes the images to prepare them for image classification.

Chapter 4: System Design and Architecture

4.1 Overview and Assumptions

This section introduces the assumptions made for each module in Accelera and explains why these assumptions were needed during the implementation.

4.1.1 Accelera Pipeline Assumptions

The system combines Python-facing APIs with native execution components. The design assumes that machine-learning workflows can be represented as a graph of operations and that only a restricted, analyzable subset of custom source code should be translated to parallel C++.

4.1.2 Auto Preprocessing Assumptions

For the AutoPreprocessing module, the assumptions depend on the type of dataset being processed. The module supports three types of data: tabular, text, and image datasets.

4.1.2.1 Tabular Dataset Assumptions

For tabular datasets, the AutoPreprocessing module assumes that:

- The dataset consists of numerical and categorical features, and the task is either classification or regression. Because we try to implement the standard structured machine learning problems where supervised learning is applied..
- The user provides configuration parameters to control preprocessing operations. This allows the pipeline to adapt to different datasets and user requirements.

4.1.2.2 Text Dataset Assumptions

For text datasets, the AutoPreprocessing module assumes that:

- The dataset contains a single text column and is intended for a text classification task with a defined target label. Because This simplifies the pipeline to standard supervised NLP classification problems and prevents the feature space from becoming very large after transforming the text into a numerical representation, making it suitable for classical machine learning techniques.

- The user provides configuration parameters for text cleaning and feature extraction. To give the user a level of control over the preprocessing pipeline, such as limiting the maximum number of numerical features generated after transforming the text into a numerical representation.

4.1.2.3 Image Dataset Assumptions

For image datasets, the AutoPreprocessing module assumes that:

- The dataset is intended for image classification and follows a folder-based class structure. Because there are many vision tasks, but we implemented the most common one to apply its processes.
- All images are stored in supported formats and are accessible. This ensures compatibility with image loading and preprocessing libraries.
- The user provides configuration parameters for preprocessing and augmentation. Because different datasets require different preprocessing strategies such as resizing, normalization, and augmentation.

The dataset is expected to follow the directory structure shown below, where the validation folder is optional:

```
Training/
  Cats/
  Dogs/

Validation/
  Cats/
  Dogs/
```

4.1.3 Benchmark Assumptions

For the Benchmark module, the following assumptions are made. These assumptions are introduced due to system design constraints, particularly the limited server storage capacity for handling large dataset uploads, as well as to ensure fair and consistent evaluation and comparison between users.

- When the user creates a benchmark, they provide a tabular dataset. This ensures that the module operates on structured data suitable for standard machine learning evaluation.
- All datasets are stored in publicly accessible Google Drive links. This design choice is adopted to reduce server storage requirements.
- The user must provide three CSV files:

  - A training dataset containing both features and target labels.
  - A testing dataset containing the same feature structure as the training set but without the target column.
  - A ground truth target file containing only the target labels for the test set.

  This separation ensures a standard evaluation protocol and prevents data leakage during benchmarking.

- When submitting predictions, the user provides a Google Drive link to a CSV file containing the predicted results. This ensures consistency in submission format across all users.
- The submitted prediction file must have the same length and format as the expected output to ensure correct evaluation and fair comparison between different users' models.
- When creating a benchmark, it is assumed that the user selects an evaluation metric that is appropriate for the problem type and provides a correct metric configuration. This is necessary to guarantee meaningful and valid performance measurement.
- For admin users, when defining an evaluation metric, it is assumed that a valid metric from the Scikit-learn library is selected. Because we use Scikit-learn metrics for evaluation.
- The selected metric must be properly configured and is expected to return a single scalar value for performance comparison and ranking purposes.
- The user is allowed to have only one active submission per benchmark. The user cannot create multiple submissions, but they are allowed to update their existing submission. This constraint is enforced to reduce storage usage.

4.2 System Architecture

4.2.1 Block Diagram

The implemented system is organized around graph execution, source-code parallelization, automated preprocessing, model-selection, deployment, and benchmarking components. The following module sections describe the material available in the previous thesis.

4.3 Accelera Pipe Module

4.3.1 Module Responsibility

Accelera Pipe is implemented under `accelera/src/accelera_pipe/`. Its responsibility is to provide a Python builder API over a C++ graph backend. The user writes a pipeline in Python, but the graph structure, node scheduling, branch execution, and intermediate memory lifetime are handled by the native backend.

The core files are:

- `core/pipeline_base.py`: owns or receives a native `graph.Graph` instance and provides shared save/load behavior.
- `core/pipeline.py`: implements the public builder API: `preprocess`, `model`, `predict`, `metric`, `merge`, and `branch`.
- `core/executed_graph.py`: wraps a trained graph for repeated inference.
- `core/metric.py`: defines the metric abstraction.
- `wrappers/`: contains supervised and unsupervised metric adapters.

4.3.2 Graph Execution Model

Each pipeline operation becomes a graph node. A preprocessing node transforms the input matrix, a model node fits or applies an estimator, a prediction node performs inference, a metric node evaluates the prediction, and a merge node combines multiple branch outputs. The graph is built incrementally, so the Python API remains fluent while the backend receives enough structure to execute independent branches in parallel.

Table 4.1: Accelera Pipe builder methods

Method	Node type	Main role
`preprocess(name, func)`	PREPROCESS	Transform input data
`model(name, model)`	MODEL	Fit or apply estimator
`predict(name, test_data)`	PREDICT	Run model prediction
`metric(name, metric)`	METRIC	Score predictions
`merge(name, method)`	MERGE	Combine branch outputs
`branch(name, ...)`	Multiple	Create alternative graph paths

The pipeline returns two objects after training: the metric or prediction results and an `ExecutedGraph`. The executed graph contains the fitted objects required for inference. This separation is important because training uses temporary arrays, validation targets, and candidate branches, while inference should keep only the selected executable state.

Algorithm 4.1: Pipeline training and executed graph construction

```
Input: training data X, target y, graph G, selection strategy S
Output: results R, executed graph E

1:  Attach X and y to the input node of G
2:  Execute ready preprocessing, model, prediction, metric, and merge nodes
3:  For each candidate path:
4:      collect validation metric and path metadata
5:  Select path P according to strategy S
6:  Clone the selected fitted nodes from P into a compact executed graph E
7:  Remove temporary training arrays and non-required execution data from G
8:  Return results R and executed graph E
```

4.3.3 Branching and Node Sharing

Branching is one of the main reasons Accelera Pipe uses a graph rather than a linear pipeline object. A user may compare several preprocessors, several models, or complete candidate sub-pipelines. The backend can then schedule independent branches concurrently. A key optimization added to the branch construction step is duplicate first-node sharing. If a branch starts with the same single preprocessing object as another branch, the graph creates the node once and connects the later branch consumers to the same output. This avoids running identical first-stage transformations multiple times.

The sharing rule is intentionally conservative. It is applied only to single branch nodes, the first node of a branch list, or a list containing one item. Later duplicated nodes inside two different lists are not merged automatically, because by that point their inputs may already be different. For example, `[p1, p2, p3]` and `[p4, p2, p3]` do not share `p2`; the first path feeds `p2` with `p1(X)`, while the second feeds it with `p4(X)`.

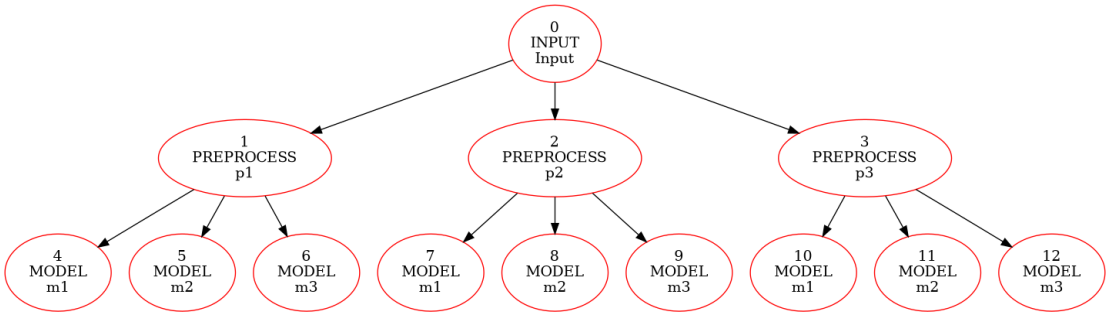

Figure 4.1: Branch graph before duplicate first-node sharing

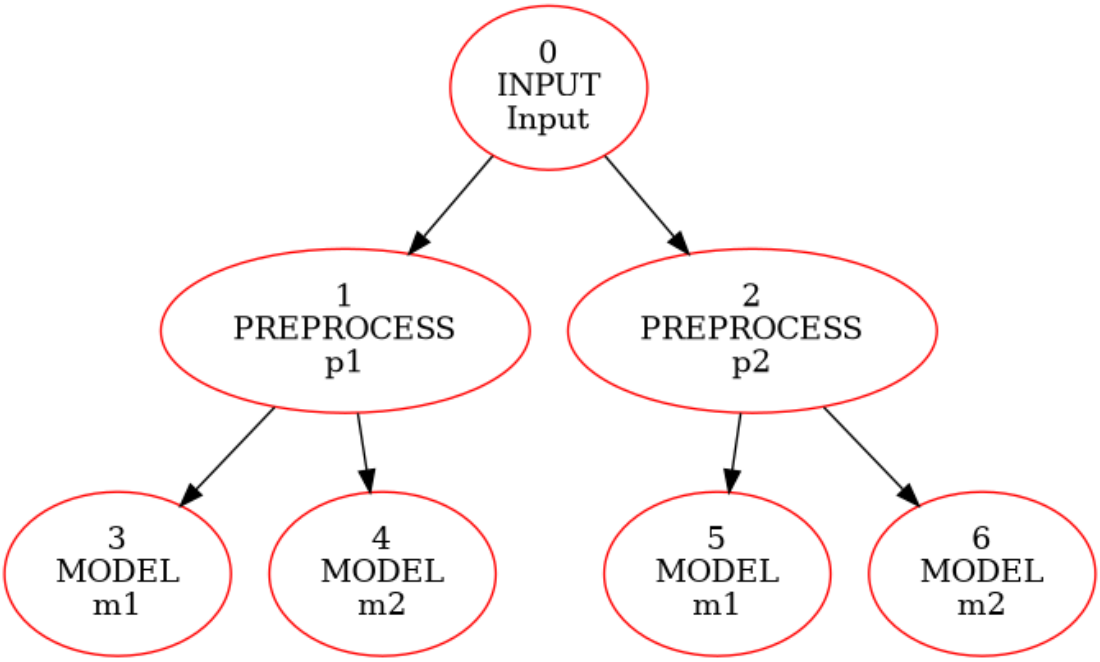

Figure 4.2: Branch graph after duplicate first-node sharing

4.3.4 Memory-Lifetime Optimizations

The largest memory pressure in Accelera Pipe appears during branch execution. Each preprocessing node can produce a transformed training array, and that array may be consumed by several downstream model nodes. The backend therefore tracks how many consumers still need each intermediate output. When the last consumer finishes, the output becomes dead and can be released.

Algorithm 4.2: Consumer-count based intermediate release

```
Input: executed node N, graph consumer count table C
Output: graph with dead temporary arrays cleared

1:  for each source node S that feeds N do
2:      C[S] = C[S] - 1
3:      if C[S] == 0 and S is releasable then
4:          clear temporary data stored by S
5:      end if
6:  end for
7:  if N is a model node and fit has completed then
8:      clear the training input stored only for fitting N
9:  end if
```

Several smaller optimizations support this lifetime policy. Cache execution is optional and disabled by default using `cache=False`, because retaining node data can increase memory growth on large datasets. Model nodes release their training arrays after fitting because the fitted estimator is sufficient for inference. The backend also separates output-container allocation from input copying. For preprocessing,

the helper `shouldCopyPreprocessInput` decides whether a defensive copy is needed. Scikit-learn transformers normally return transformed data without mutating the original input, so unnecessary copies can be skipped for those objects. Custom functions remain protected when mutation is possible.

4.3.5 Saving and Loading Pipelines

Pipeline objects can be saved as pickle files. Native graph pickling is provided through the PyBind binding for `graph.Graph`, where C++ exposes a `py::pickle` state pair. During `pickle.dump`, Python asks the bound object for its registered pickle state, and the C++ graph returns a Python dictionary that describes the graph structure and stored objects. During load, the inverse state function rebuilds the graph object.

Custom preprocessing functions are handled by storing source code when normal pickle cannot serialize the function object directly. The save path extracts a top-level function or simple lambda source, and the load path executes that source inside a controlled namespace to recover the callable. Functions with closures are rejected because their external captured variables would not be available from source text alone.

4.4 Code Parallelizer Module

4.4.1 Module Responsibility

The Code Parallelizer transforms supported loop-heavy source code into OpenMP-annotated C++. The main Python orchestration file is `accelera/src/utils/parallelizer.py`. Python input is first lowered by `accelera/src/utils/py2cpp_converter.py`. Native loop extraction and pragma insertion are implemented in C++ under `src/ast/` and `src/utils/code_parallelizer_utils.cpp`. The module can process files or in-memory code strings.

The workflow has four stages:

1. Convert supported Python code to C++ when the input is Python.
2. Use Clang-based analysis to extract loops and loop metadata.
3. Predict the loop class using a classifier and rule checks.
4. Insert OpenMP pragmas and return the transformed C++ code.

4.4.2 Loop Analysis and Pragma Selection

Loop extraction records syntactic and semantic features such as loop nesting, array access, assignments, reduction patterns, loop-carried dependencies, function calls, and scalar updates. These features are passed to the classifier service. The classifier predicts one of three classes: `none`, `parallel_for`, or `reduction`. The rule layer then validates the predicted class before emitting the pragma.

Algorithm 4.3: Loop classification and OpenMP annotation

```
Input: source program P
Output: OpenMP-annotated C++ program P'

if P is Python then
    convert the supported Python subset to C++
end if
Parse C++ source using the Clang frontend action
for each extracted loop L do
    compute structural and dependency features F(L)
    request class c from the OpenMP classifier
    if c is parallel_for and dependency checks pass then
        insert "#pragma omp parallel for"
    else if c is reduction and reduction variable is valid then
        insert "#pragma omp parallel for reduction(...)"
    else
        keep the loop sequential
    end if
end for
Return the rewritten C++ source
```

4.4.3 Classifier Model and Rule-Based Decision

The classifier assets are stored under `models/open_mp_classifier/`. The service loads `model.pkl` and `label_encoder.pkl`, exposes a FastAPI prediction endpoint, and maps loop features to one of the supported pragma classes. The trained model is an `XGBClassifier` over a fixed 40-feature loop representation.

An earlier generator trial attempted to generate pragma decisions using a neural sequence model. The model used a custom PyTorch encoder-decoder architecture: a bidirectional LSTM encoder, an attention-based LSTM decoder, teacher forcing during training, cross-entropy loss, Adam optimization, gradient clipping, and a custom BPE tokenizer. This approach was rejected for two scientific reasons. First, the available dataset contained about 15,000 samples, which was not enough for a sequence model to generalize reliably to unseen program structures; the model overfit easily. Second, generating extra training programs with AI tools risked injecting generator-specific bias, which would lower generalization on real user code. The final approach therefore uses explicit loop features, a classifier, and rule-based safety checks.

4.4.4 Python-to-C++ Converter Support

The Python converter intentionally supports a restricted subset. This makes the generated C++ predictable enough for the parallelizer and avoids silently miscompiling dynamic Python behavior.

Table 4.2: Supported operations in the Python-to-C++ converter

Category	Supported forms	Notes
Values and names	Numeric constants, names, simple lists	Dynamic objects are outside the subset.
Arithmetic	`+`, `-`, `*`, `/`, `%`, `**`	Power maps to `std::pow`.
Boolean logic	`and`, `or`, `not`	Emitted as C++ boolean expressions.
Comparisons	`<`, `<=`, `>`, `>=`, `==`, `!=`	Chained comparisons are restricted.
Calls	Supported built-ins and direct calls	Unsupported calls raise converter errors.
Indexing	Array-style subscripting	Slice semantics are not supported.
Assignment	Simple assignment and selected augmented assignment	Used for loop updates and reductions.
Control flow	`if/else`, `return`, `for range(...)`	Main loop form used by the parallelizer.
Functions	Plain function definitions	Decorators, closures, and dynamic dispatch are outside the safe subset.

4.4.5 Integration with Accelera Pipe

The parallelizer is integrated into Accelera Pipe for custom preprocessing functions and custom transformer instances. If a preprocessing function is a user-defined Python function, the pipeline attempts to optimize it through the parallelizer. If the object is a custom transformer, the transform method can be optimized and attached back to the instance. This allows the user to write a normal Python preprocessing step while the system compiles the loop-heavy part into a Python-callable C++ extension.

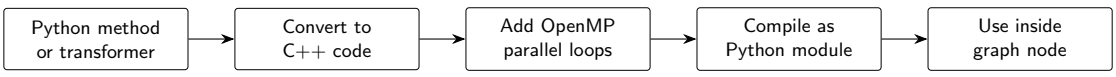

Figure 4.3: Integration flow for compiled preprocessing inside Accelera Pipe

The integration is applied during graph construction rather than as a separate user-facing execution stage. A custom preprocessing method is inspected, lowered to the supported C++ subset, analyzed for loop-level parallelism, compiled into a Python-callable native module, and then stored back inside the preprocessing node. As a result, the graph still executes a normal preprocessing node, but the callable held by that node may be a compiled OpenMP implementation instead of the original Python loop.

4.5 AutoPreprocessing Module

The AutoPreprocessing module is responsible for automating the data understanding, exploratory data analysis, and preprocessing stages of the data science workflow. The module was designed to reduce repeated manual work and to prepare different types of datasets in a consistent way before model training.

4.5.1 Functional Description

The main function of this module is to receive a dataset and user configuration, then return processed data that is ready for machine learning or deep learning models. The module supports tabular, text, and image datasets.
For tabular datasets, the module supports both classification and regression tasks. It analyzes the dataset, removes duplicate records, splits the data into training and validation sets, detects column types, handles missing values, encodes categorical features, scales numerical features, applies target preprocessing, performs feature selection, and saves the preprocessing objects needed later for test data.
For text datasets, the module supports classification tasks. It cleans the text, tokenizes it, applies text preprocessing steps, and converts the text into a numerical representation suitable for machine learning models. For image datasets, the module supports image classification by reading images from class folders, validating image files, resizing images, normalizing pixel values, and preparing the data loaders needed for training.
The module also generates a report when this option is enabled. The report summarizes information about the dataset and the preprocessing steps applied. For example, in tabular data, it includes dataset shape, column types, missing values, duplicate analysis, generated graphs, removed columns, and previews of the processed data.

4.5.2 Modular Decomposition

The AutoPreprocessing module is decomposed into smaller components according to the dataset type and the stage of preprocessing.

4.5.2.1 Input and Configuration Validation

This component receives the user inputs, such as the dataset, target column, problem type, output folder, validation size, random state, missing-value threshold, cardinality threshold, and report flag. It validates these values before running the pipeline to prevent invalid configurations, such as a wrong target column, unsupported problem type, or invalid threshold values.

4.5.2.2 Data Understanding and Reporting

This component collects the first information about the dataset before preprocessing. It stores dataset previews, column data types, missing values, duplicate count, dataset shape, numerical statistics, and categorical statistics. These results are saved in a report structure and later used to generate the final preprocessing report.

4.5.2.3 Tabular Preprocessing

This component handles structured data organized in rows and columns. It first removes duplicates, then splits the dataset into training and validation sets. After that, it detects feature types such as binary, ordinal, numerical, continuous, low-cardinality categorical, high-cardinality categorical, and unsupported columns.
Each feature type has its own preprocessing pipeline. Numerical and continuous features use median imputation followed by robust scaling. Binary features use most-frequent imputation followed by ordinal encoding. Low-cardinality categorical features use most-frequent imputation followed by binary encoding. High-cardinality categorical features use most-frequent imputation followed by frequency encoding. Unsupported columns and columns above the missing-value threshold are removed.

4.5.2.4 Target Preprocessing

This component prepares the target column. For classification, missing target values are filled using the most frequent class, then the target is transformed using label encoding. For regression, missing target values are filled using the median, then robust scaling is applied to the target values.

4.5.2.5 Feature Selection

After preprocessing the input features, mutual information is used to select the most informative features. For classification, mutual information classification is used. For regression, mutual information regression is used. Only features with scores greater than or equal to the selected threshold are kept.

4.5.2.6 Text Preprocessing

This component handles text classification datasets. It validates the text configuration, cleans the text, applies tokenization and text normalization, and converts the processed text into numerical features. The generated preprocessing objects are saved so the same transformations can be applied to testing data.

4.5.2.7 Image Preprocessing

This component handles image classification datasets. Images are collected from class folders and assigned labels based on their folder names. The system checks

whether images can be loaded correctly, removes invalid images, resizes valid images, normalizes pixel values, and applies optional augmentation such as flipping, rotation, brightness adjustment, and contrast adjustment.

4.5.2.8 Artifact Saving

This component saves the objects required to reproduce the same preprocessing steps later. Examples include selected columns, dropped columns, feature names, target preprocessor, training preprocessor, class mappings, and report data. This makes the preprocessing pipeline reusable for validation, testing, or future prediction.

4.5.3 Design Constraints

Several constraints affected the design of the AutoPreprocessing module:

- The module must avoid data leakage. For this reason, preprocessing objects are fitted only on the training data and then applied to validation or testing data.
- The module must support different data types. Therefore, separate preprocessing paths were designed for tabular, text, and image datasets.
- The tabular path assumes that the data is structured in rows and columns and that the task is either classification or regression.
- The text path assumes that the dataset contains a text column and a target label for a text classification task.
- The image path assumes that image classification data follows a folder-based class structure, where each folder represents one class.
- The module must keep preprocessing reproducible. Therefore, fitted preprocessing objects and selected features are saved for later use.
- The module must be general enough for different datasets, so the design uses rule-based decisions instead of manual preprocessing code for each dataset.

4.5.4 Other Description of the Module

The AutoPreprocessing module is not only designed to transform data. It is also designed to help the user understand the dataset before and after preprocessing. This is achieved through the generated report and visualizations. For tabular data, the report can show missing-value statistics, duplicate information, feature distributions, target distribution, correlation matrix, removed columns, preprocessing decisions, and previews of the final processed training and validation data.
Another important design point is that the module separates training preprocessing from testing preprocessing. During training, the module fits the preprocessing objects and saves them. During testing, the saved objects are loaded and applied directly, which ensures that new data is transformed in the same way as the training data.

4.6 AutoML Module

4.7 Deployment Module

4.8 Benchmark Platform Module

The Benchmark Platform module is designed to provide a shared environment where users can create machine learning benchmarks, submit predicted results, and compare submissions using selected evaluation metrics. The module includes a backend API, database schemas, Python validation and scoring scripts, and a frontend interface for users and admins.

4.8.1 Functional Description

The main function of this module is to support fair comparison between different machine learning solutions. A user can create a benchmark by providing the benchmark title, description, problem type, target column, dataset link, test dataset link, ground-truth target link, and evaluation metric. The system validates the provided links and checks that the test set does not contain the target column while the target file contains the expected target column.
After a benchmark is created, other users can submit their prediction file and repository link. The system validates the submitted prediction file, calculates the score using the selected Scikit-learn metric, stores the submission, and displays submissions on a leaderboard. The leaderboard is ordered based on the metric configuration, where some metrics are better when their value is higher and others are better when their value is lower.

4.8.2 Modular Decomposition

The Benchmark Platform is decomposed into several smaller components:

4.8.2.1 Benchmark Management

This component handles benchmark creation, listing, filtering, viewing, and deletion. It stores benchmark information such as title, description, problem type, dataset links, target column, selected metric, metric parameters, creation date, and creator.

4.8.2.2 Metric Management

This component allows admin users to define evaluation metrics. Each metric contains a display name, the corresponding Scikit-learn metric function name, needed parameters, problem type, and whether a higher or lower score is better. Metrics are linked to benchmarks and used during submission scoring.

4.8.2.3 Submission Management

This component allows users to submit prediction files for a specific benchmark. Each submission stores the benchmark id, submitting user, repository link, prediction file link, score, and submission date. Users can also update their existing prediction file or delete their submission if they have permission.

4.8.2.4 Validation and Scoring Scripts

The backend calls Python scripts to validate benchmark files and score submissions. During benchmark creation, the validation script checks that the test file does not include the target column and that the target file contains the required target column. During submission, the scoring script loads the ground-truth file and the submitted prediction file, checks their length and target column, then computes the selected Scikit-learn metric.

4.8.2.5 Leaderboard Interface

The frontend displays benchmark submissions in a leaderboard. It shows the submitter information, repository link, score, and submission date. The sorting direction depends on the selected metric, so the platform can support both metrics where higher is better and metrics where lower is better.

4.8.3 Design Constraints

Several constraints affected the design of the Benchmark Platform:

- The module currently supports classification and regression problem types only.
- Dataset, test, target, and prediction files are provided using links, mainly Google Drive file links, to reduce server storage requirements.
- The test dataset must not contain the target column. This prevents data leakage and makes the evaluation fair.
- The target file and submitted prediction file must contain the selected target column and must have the same number of rows.
- Evaluation metrics must be defined using valid Scikit-learn metric names because the scoring script calls Scikit-learn metrics dynamically.

- Only authenticated users can create benchmarks or submit predictions. Admin permission is required for metric creation.
- A user is allowed to create only one active submission for the same benchmark. If the user wants to change the result, the existing submission is updated instead of creating duplicate submissions.

4.8.4 Other Description of the Module

The Benchmark Platform separates benchmark creation from submission evaluation. This makes the workflow clearer: first, the benchmark creator defines the task and evaluation rules; then, users submit predictions under the same conditions. This separation helps ensure that all users are evaluated using the same ground-truth file and the same metric configuration.
Another important design point is the use of Python scripts for file validation and metric scoring. The backend is implemented using Node.js and Express, while the scoring process uses Python because it depends on Scikit-learn metrics and dataset loading utilities already used in the project. This design allows the web platform to manage users, benchmarks, and submissions while Python handles the machine learning evaluation part.

Chapter 5: System Testing and Verification

5.1 Testing Setup

The available experiments use identical train, validation, and test splits when comparing candidate pipelines. Timing, memory, output equivalence, and predictive accuracy are reported according to the module under test.

5.2 Testing Plan and Strategy

Module tests validate graph correctness, pipeline persistence, converter support, loop transformation, and output equivalence. Integration tests evaluate the end-to-end path from custom preprocessing code through native compilation and graph execution.

5.3 Accelera Pipe Module

The Accelera Pipe experiments compare three implementations under the same train, validation, and test split. The first implementation is a manually written scikit-learn loop over the candidate pipelines. The second uses `sklearn.pipeline.Pipeline` to represent the same candidates. The third uses Accelera Pipe with graph branches. Each candidate is selected using validation accuracy, and the final selected path is evaluated on the test set. This protocol checks whether the graph execution improves latency without changing the selected model or the measured predictive accuracy.

Table 5.1: Accelera Pipe latency scaling in seconds

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	0.06	0.05	0.34
5,000	2.47	2.12	2.02
10,000	2.48	2.48	1.79
50,000	23.96	23.25	14.81
100,000	77.79	77.10	67.04
250,000	803.56	803.01	494.64

Table 5.1 shows the main performance trend without compressing the font. At very small sizes, Accelera Pipe is slower because the native graph setup, branch creation, and Python/C++ boundary cost dominate the actual model computation. Once the dataset grows, those fixed costs become small compared with fitting multiple candidate

branches. At 50,000 samples, Accelera Pipe reduces runtime from 23.96 seconds to 14.81 seconds. At 250,000 samples, it reduces runtime from about 803 seconds to about 495 seconds, saving more than five minutes while selecting the same candidate.

Table 5.2: Accelera Pipe RSS memory scaling in MB

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	1.62	0.16	19.32
5,000	3.62	1.09	21.62
10,000	11.47	2.19	7.25
50,000	126.95	9.75	209.25
100,000	139.70	47.65	300.93
250,000	303.95	28.52	594.86

Table 5.2 shows the current cost of graph-level parallelism. Accelera Pipe uses more RSS memory because multiple branch states and intermediate arrays can exist at the same time. The memory optimizations in the methodology section, especially consumer-count based release, duplicate first-node sharing, and guarded preprocessing copies, target exactly this issue.

Table 5.3: Selected candidate and accuracy across sample sizes

Samples	Selected candidate	Validation accuracy	Test accuracy
1,000	MinMaxScaler $\rightarrow$ SVC	0.7900	0.8250
5,000	MinMaxScaler $\rightarrow$ SVC	0.9160	0.9100
10,000	MinMaxScaler $\rightarrow$ SVC	0.9385	0.9260
50,000	MinMaxScaler $\rightarrow$ SVC	0.9565	0.9598
100,000	StandardScaler $\rightarrow$ SVC	0.9675	0.9709
250,000	StandardScaler $\rightarrow$ SVC	0.9774	0.9768

Table 5.3 confirms that the speedup does not come from changing the search space. Accelera Pipe evaluates the same candidate pipelines and reports the same validation and test behavior. The selected preprocessing strategy changes from MinMax scaling to standard scaling only when the larger dataset makes StandardScaler with SVC the best validation candidate. This is expected because the candidate set and selection rule are fixed while the data distribution becomes better represented at larger sizes.

Table 5.4: Largest Accelera Pipe comparison

Implementation	Time (s)	RSS MB	VMS MB	Test accuracy
sklearn manual loop	803.56	303.95	315.41	0.9768
sklearn Pipeline	803.01	28.52	28.61	0.9768
Accelera Pipe	494.64	594.86	787.47	0.9768

Table 5.4 isolates the largest run: 150,000 training samples, 50,000 validation samples, and 50,000 test samples. Accelera Pipe keeps exactly the same selected path, StandardScaler followed by SVC, and the same test accuracy. The runtime drop is 308.91 seconds relative to the manual scikit-learn loop and 308.36 seconds relative to scikit-learn Pipeline. This result is important because it demonstrates the value of graph scheduling on realistic branch-heavy workloads. The memory columns also show the next engineering target: scikit-learn Pipeline is extremely memory efficient, while Accelera Pipe trades memory for parallel branch execution and native graph state.

5.4 Code Parallelizer Module

The OpenMP classifier was evaluated as a three-class loop classifier. The labels are `none`, `parallel_for`, and `reduction`. The classification report shows that the model is balanced enough to guide the rule layer, while the final pragma decision remains protected by dependency checks.

Table 5.5: OpenMP classifier report

Class	Precision	Recall	F1-score	Support
none	0.82	0.85	0.84	3048
parallel_for	0.86	0.81	0.83	2696
reduction	0.79	0.82	0.80	775
accuracy			0.83	6519
macro avg	0.82	0.83	0.83	6519
weighted avg	0.83	0.83	0.83	6519

Table 5.5 should be read as a guidance-quality result, not as a proof that every predicted loop is safe. The classifier is responsible for identifying likely parallelization opportunities. The rule layer is responsible for rejecting unsafe or unsupported cases. This two-stage design is useful because source code parallelization has correctness constraints that pure statistical classification should not decide alone.

Table 5.6: Hard parallelizer evaluation

File	Baseline ms	Parallel ms	Speedup	Output match
hard_eval_000.cpp	1786.28	195.47	9.138×	true
hard_eval_001.cpp	63.98	31.47	2.033×	true
hard_eval_002.py	42.66	7.67	5.563×	true

Table 5.6 evaluates harder programs from `data/hard_test_files`. The benchmark compiles and runs both the baseline and the generated parallelized code, then compares their outputs. The first file obtains the largest improvement because its loop body has enough work to amortize OpenMP thread overhead. The second file still improves, but the smaller absolute runtime limits the maximum speedup. The Python-origin file demonstrates the full converter path: Python subset to C++, then loop extraction, then pragma insertion. Across the three files, the evaluation extracted 11 loops, generated 8 pragmas, matched all 8 expected pragmas, reached average similarity 1.0, and obtained an average speedup of 5.578×.

Table 5.7: Linux parallelizer and Accelera Pipe integration benchmark

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.83
End-to-end optimized run, first measured process	500,000	5.88
End-to-end optimized run, warmed method and module cache	500,000	0.08

Table 5.7 reports the Linux direct `examples/parallel_accpipe.py` run after normalizing the example execution directory to the repository root. The Python implementation takes 6.83 seconds on 500,000 samples. The first optimized process in the measurement sequence takes 5.88 seconds because it still pays process-local setup and cache lookup costs. The immediate repeated run takes 0.08 seconds after the Python-method cache and compiled-module cache are both warm. The validation in the example confirmed that the optimized output matched the Python output. This result shows why the integration needs two cache layers: one layer avoids repeating Python-to-C++ conversion and OpenMP insertion, while the lower compiled-module cache avoids recompiling the pybind extension.

Table 5.8: Linux direct example-script verification runs

Example	Baseline time (s)	Optimized time (s)	Validation
`accpipe_demo`	2.48	1.79	same accuracy, 0.9260
`parallel_accpipe`, run 1	6.83	5.88	outputs match
`parallel_accpipe`, run 2	6.83	0.08	outputs match
`code_parallelizer`, run 1	5.81	0.014	outputs match
`code_parallelizer`, run 2	6.05	0.015	outputs match
`save_load`, run 1	1.89	2.69 / 2.65	same accuracy, 0.268
`save_load`, run 2	2.53	2.20 / 2.14	same accuracy, 0.268

Table 5.8 summarizes the Linux examples invoked by `examples/Makefile`, but each file was run directly rather than through Make. Cache-sensitive files were repeated. The standalone `code_parallelizer_demo.py` result measures Python script execution against the generated OpenMP executable for `hard_eval_002.py`; the C path also reported about 0.26–0.27 seconds of compilation time, outside the listed executable runtime. The save/load example verifies persistence rather than pure acceleration: both sklearn and Accelera load persisted state and produce the same accuracy, while the second Accelera run is slightly faster than the sklearn baseline.

Table 5.9: Windows direct example correctness and path timing

Example	Run/cache state	Baseline (s)	Accelera/C path (s)	Validation
`accpipe_demo`	normal	1.75 / 1.78	1.03	same test accuracy, 0.9260
`parallel_accpipe`	isolated cache, run 1	6.66	6.00	outputs match
`parallel_accpipe`	isolated cache, run 2	6.58	0.09	outputs match
`code_parallelizer`	isolated cache, run 1	5.59	1.04	outputs match
`code_parallelizer`	isolated cache, run 2	5.61	1.05	outputs match
`save_load`	normal	0.57	0.51 / 0.53	same accuracy, 0.268

Table 5.10: Windows compilation and process overhead

Example	Run/cache state	Compile/link (s)	Wall time (s)	Interpretation
`accpipe_demo`	normal	–	6.53	graph example startup
`parallel_accpipe`	isolated cache, run 1	included	14.37	cold compile path
`parallel_accpipe`	isolated cache, run 2	cached	8.33	warm cache path
`code_parallelizer`	isolated cache, run 1	0.87	8.74	temporary executable
`code_parallelizer`	isolated cache, run 2	1.04	9.00	temporary executable
`save_load`	normal	–	2.71	persistence example

Tables 5.9 and 5.10 report the same Makefile example set on Windows. The first table keeps the user-visible comparison: baseline runtime, optimized path runtime, and validation result. The second table separates compilation and process-level overhead so the timing table remains readable. The examples were run one by one

in Makefile order. The cache-sensitive examples were then repeated under an isolated Accelera cache so that the first run represents cold cache behavior and the second run represents warm cache behavior. The strongest cache effect appears in `parallel_accpipe.py`: the cold end-to-end Accelera path takes 6.00 seconds because it must convert the function, select loops, compile a pybind extension, link it, and load it into Python. The immediate warm-cache run takes 0.09 seconds because the processed-code cache, method cache, and compiled-module cache avoid repeating those setup stages.

The Windows `code_parallelizer_demo.py` runs show a different limitation. The generated C/OpenMP executable is compiled into a temporary directory on every run, so the second run does not reuse the final linked program. The measured C executable subprocess takes about 1.04 seconds, while the instrumented main-body timer inside the generated program reports only about 0.024–0.025 seconds. This gap is Windows process startup, OpenMP runtime initialization, dynamic library loading, and executable loading overhead. The compile/link step itself adds about 0.87–1.04 seconds. On Linux, the same benchmark has lower launch and linking overhead in the reported measurements, so executable runtime is closer to the timed main-body work. For this reason, Windows results should be interpreted as end-to-end toolchain latency results, not only as loop-kernel speed results.

5.5 AutoPreprocessing Module

5.5.1 Experimental Setup

Experimental Environment All experiments were implemented in Python using Pandas, NumPy, and Scikit-learn for tabular data processing and model evaluation. AutoCleanML was used as the baseline preprocessing framework, while Accelera was used as the proposed automated preprocessing system. The experiments were executed under identical conditions.

Datasets A total of 16 tabular datasets were used, collected from Kaggle. The datasets include both classification and regression tasks with varying sizes and feature complexities.

Table 5.11: Summary of Datasets Used in Experiments

Dataset	Shape	Task Type
startup_founder_burnout_2026	(50000, 29)	Classification
healthy_diet_calorie_intake	(6000, 15)	Classification
ai_student	(50000, 16)	Classification
gaming_addiction	(250, 49)	Classification
Teen_Mental_Health_Dataset	(1200, 13)	Classification
student_exam_performance_dataset	(10000, 17)	Classification
heart	(302, 14)	Classification
Dataset_spine	(310, 7)	Classification
diabetes_dataset	(100000, 31)	Classification
student_placement_synthetic	(100000, 18)	Classification
Titanic-Dataset	(891, 12)	Classification
customer_purchase_data	(1388, 9)	Classification
global_ai_jobs	(90000, 35)	Regression
CarPrice_Assignment	(205, 26)	Regression
Concrete_Data_Yeh	(1005, 9)	Regression
Housing	(545, 13)	Regression

Compared Frameworks Two preprocessing tools were evaluated:

AutoCleanML: A fully automated data cleaning AutoCleanML Python library, that performs missing value handling, encoding, and feature preparation.

Accelera: A proposed automated preprocessing tools supporting tabular, image, and text data, with additional reporting and visualization capabilities.

Preprocessing Configuration Identical experimental settings were used across both tools. A validation size of 20% and a random state of 42 were applied for all datasets. For classification tasks, stratified sampling was used to preserve class distribution.

Apart from specifying the target column and excluding unnecessary columns when required, all preprocessing parameters were kept at their default values for both frameworks.

Results and Discussion

Classification Results The performance of both frameworks was evaluated on multiple classification datasets using F1-macro score as the evaluation metric.

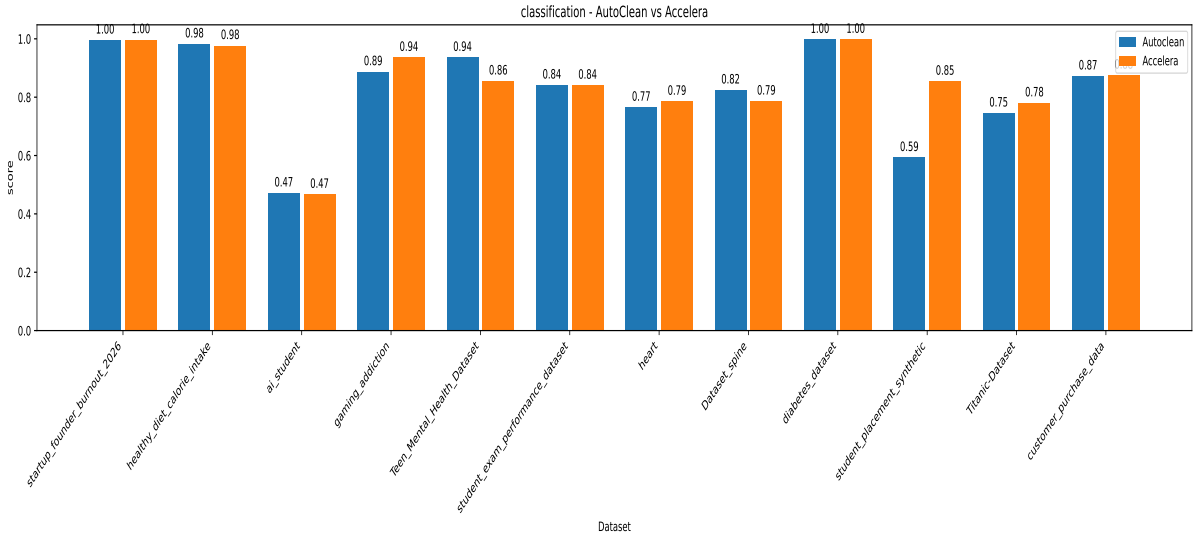

Figure 5.1: Classification Performance Comparison (F1-Macro Score)

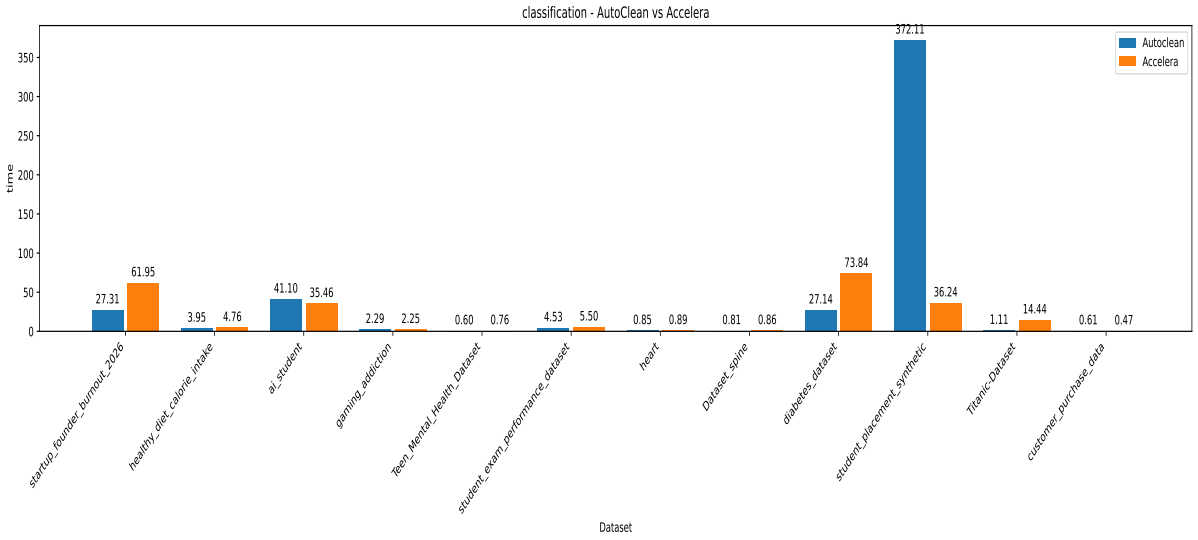

Figure 5.2: Classification Preprocessing Time Comparison

Regression Results Regression performance was evaluated using the R^2 score.

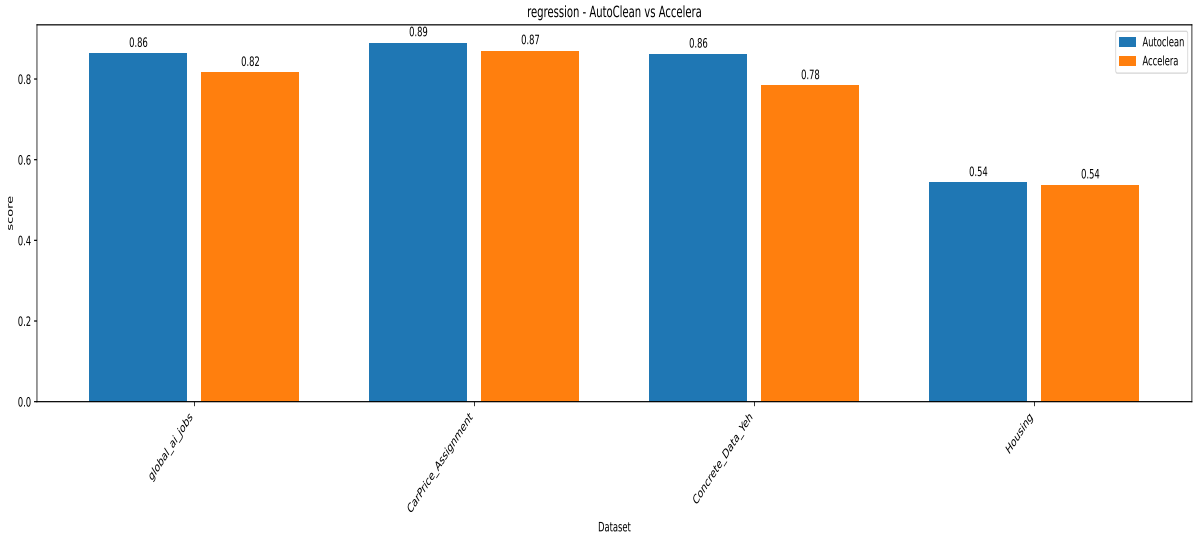

Figure 5.3: Regression Performance Comparison (R^2 Score)

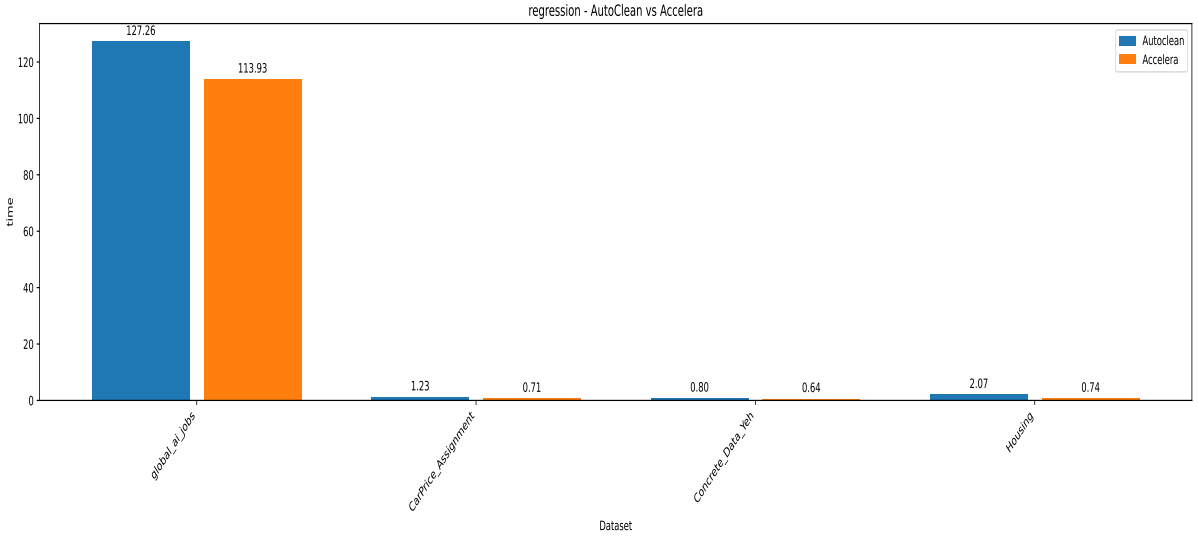

Figure 5.4: Regression Preprocessing Time Comparison

5.6 AutoML Module

5.7 Deployment Module

5.8 Benchmark Platform Module

Chapter 6: Conclusions and Future Work

6.1 Faced Challenges

The main implementation challenges were preserving correctness while sharing branch computations, releasing intermediate arrays only after their final consumer, and limiting generated OpenMP code to loops that pass dependency and conversion checks. The experiments also identify memory usage during branch-level parallel execution as the main remaining graph-backend challenge.

6.2 Gained Experience

6.3 Conclusions

Accelera Pipe provides a graph-based execution model for branch-heavy machine-learning pipelines and supports fitted executed graphs. The Code Parallelizer provides a source-to-source path from supported Python or C++ loops to OpenMP-annotated C++ using converter rules, loop feature extraction, classifier guidance, and safety checks. The available experiments show runtime improvements on large branch-selection workloads while preserving selected models, accuracy, and output equivalence.

6.4 Future Work

Future work should continue reducing graph memory consumption while preserving parallel branch execution, and should extend safe converter and loop-analysis coverage without weakening correctness checks.

References

Bibliography

[1] “Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices.” Available at: https://www.proquest.com/openview/17dfa731b6147c8939e3c5cc784897ce/1?pq-origsite=gscholar&cbl=5046920

Chapter A: Development Platforms and Tools

The implemented modules use Python, C++, PyBind11, CMake, Clang-based AST analysis, OpenMP, NumPy, and Scikit-learn. The available experiments include Linux and Windows execution paths.

Chapter B: Use Cases

Chapter C: User Guide

Chapter D: Code Documentation

Chapter E: Feasibility Study